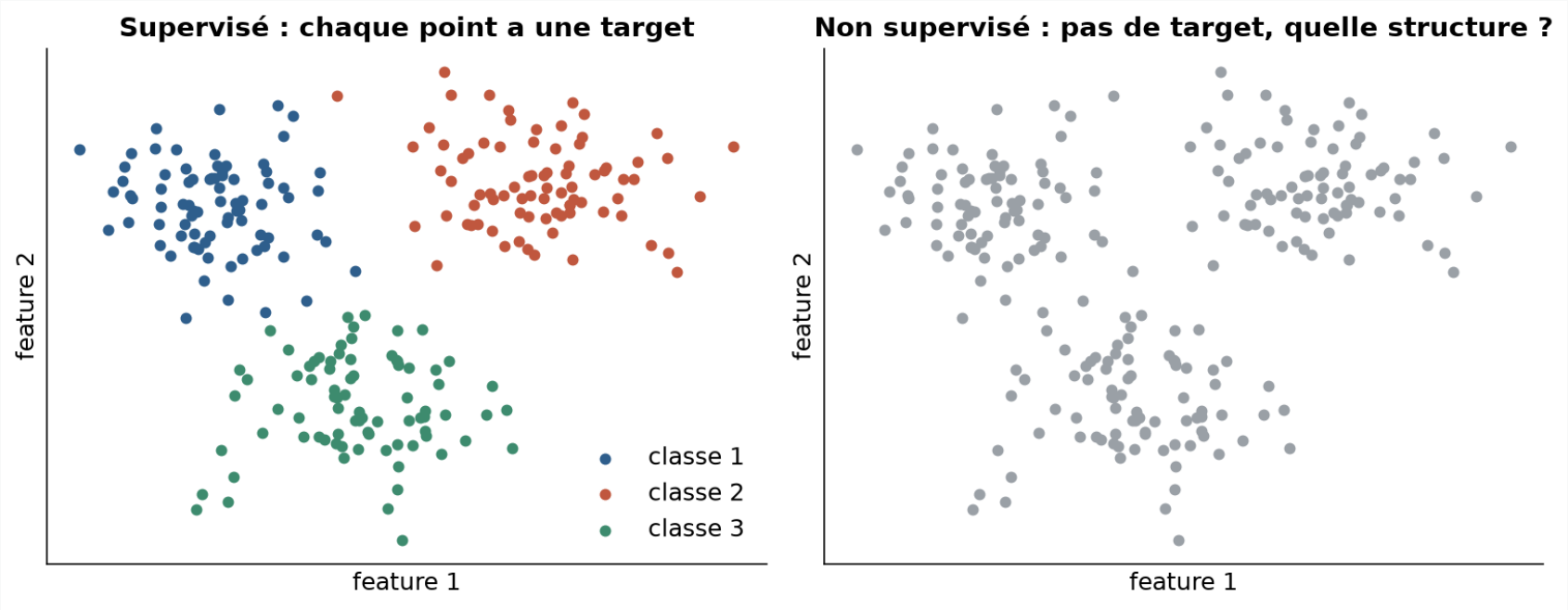
# Apprentissage non supervisé

Trouver une structure dans des données qui n'ont pas de target : regrouper (clustering), résumer (PCA), repérer l'inhabituel (anomaly detection). Ce que font ces méthodes, quand s'en servir, et comment ne pas se faire piéger.

# Jusqu'ici, chaque modèle apprenait à prédire une target. Ici, il n'y a pas de target.

On ne cherche plus à prédire une réponse connue : on cherche une **structure** dans les données. Des groupes, des directions principales, des cas qui ne ressemblent à rien.

# La même donnée, deux questions




**À gauche**, chaque point arrive avec sa classe : on apprend à la prédire, et on mesure l'erreur. **À droite**, les mêmes points sans étiquette : l'algorithme doit proposer une structure, et personne ne peut lui dire s'il a raison.

# Supervisé et non supervisé, côte à côte

| | Supervisé | Non supervisé |
|---|---|---|
| Données | features X **et** target y | features X seulement |
| Question | prédire y pour un nouveau cas | quelle structure dans X ? |
| Exemples | classification, regression | clustering, dimensionality reduction, anomaly detection |
| **Évaluation** | **accuracy, F1, RMSE sur un test set** | **pas de vérité terrain : métriques internes et jugement métier** |
| Risque principal | overfitting | voir une structure qui n'existe pas |

# Trois familles de méthodes

*Une même idée : apprendre la forme des données, sans réponse à prédire.*

**Clustering** regrouper les observations qui se ressemblent. Segmenter des clients, des patients, des documents.

**Dimensionality reduction** résumer beaucoup de features en quelques-unes. Visualiser en 2D, compresser, enlever la redondance.

**Anomaly detection** repérer ce qui ne ressemble pas au reste. Fraude, panne de capteur, erreur de saisie.

**Point commun** aucune n'a de target : **le jugement humain** fait partie de la méthode.

A

# Clustering

Regrouper ce qui se ressemble : k-means, clustering hiérarchique, DBSCAN, choisir le nombre de groupes, et évaluer sans target.

# Qu'est-ce qu'un cluster ?

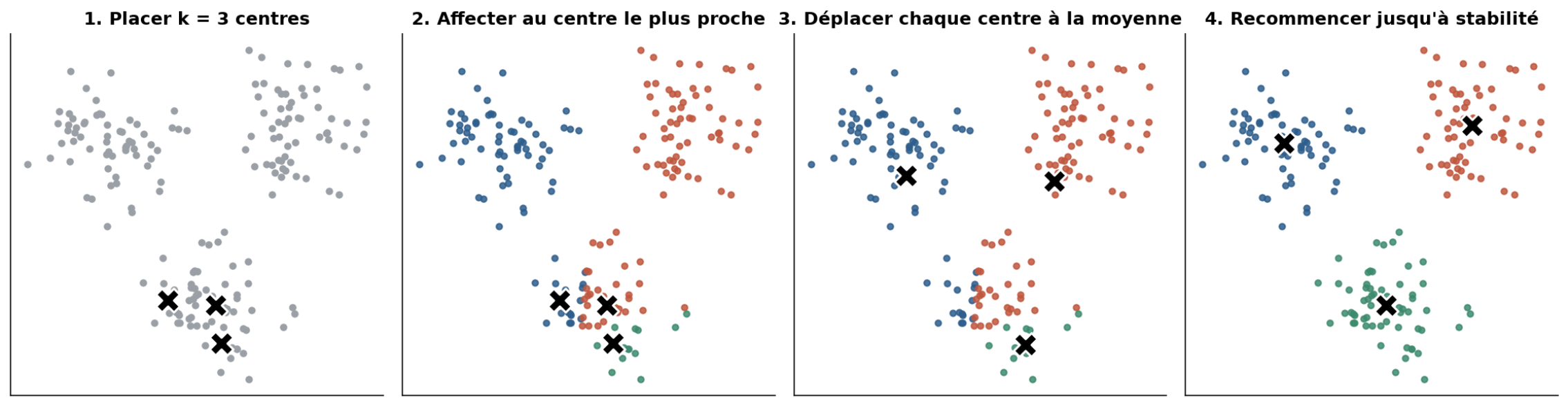
**Une idée de ressemblance** deux points se ressemblent s'ils sont **proches** : tout repose sur une distance, souvent euclidienne.

**Proche de quoi ?** d'un centre (k-means), de ses voisins de proche en proche (hiérarchique), d'une zone dense (DBSCAN).

**Pas de définition unique** chaque algorithme a **sa** définition du cluster, donc sa propre réponse sur les mêmes données.

**Conséquence** les features choisies et leur scaling décident de ce que « proche » veut dire.

# k-means en quatre gestes




On choisit **k** et on place k centres. On affecte chaque point au centre le plus proche, on déplace chaque centre à la moyenne de ses points, et on recommence jusqu'à ce que plus rien ne bouge. L'algorithme minimise l'**inertia** : la somme des distances au carré entre chaque point et son centre.

# k-means : ce qu'il faut retenir

**k est à choisir** l'algorithme ne le devine pas : c'est un **hyperparameter**.

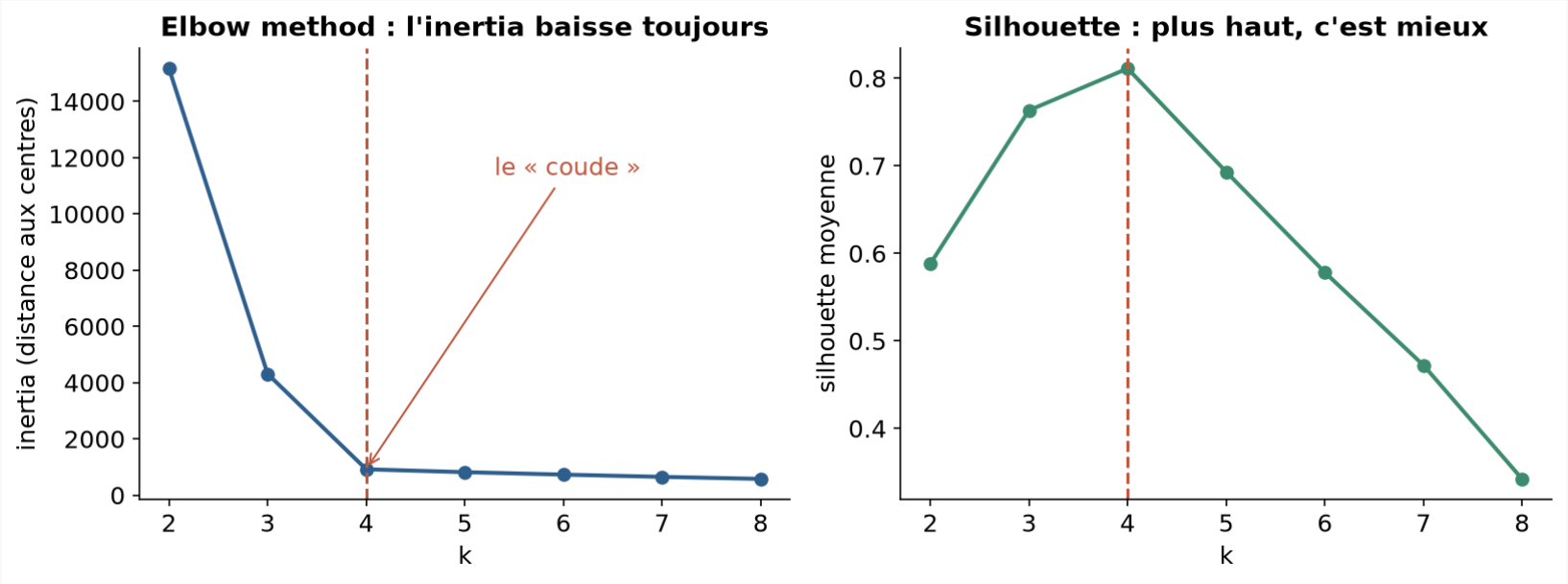
**Le départ compte** les centres initiaux sont aléatoires : on relance plusieurs fois (`n_init`) et on garde la plus faible inertia.

**Des groupes ronds** k-means suppose des groupes compacts autour d'un centre ; il découpe mal les formes allongées ou imbriquées.

**Tout point est affecté** même un point aberrant reçoit un cluster : k-means n'a pas de notion de noise.

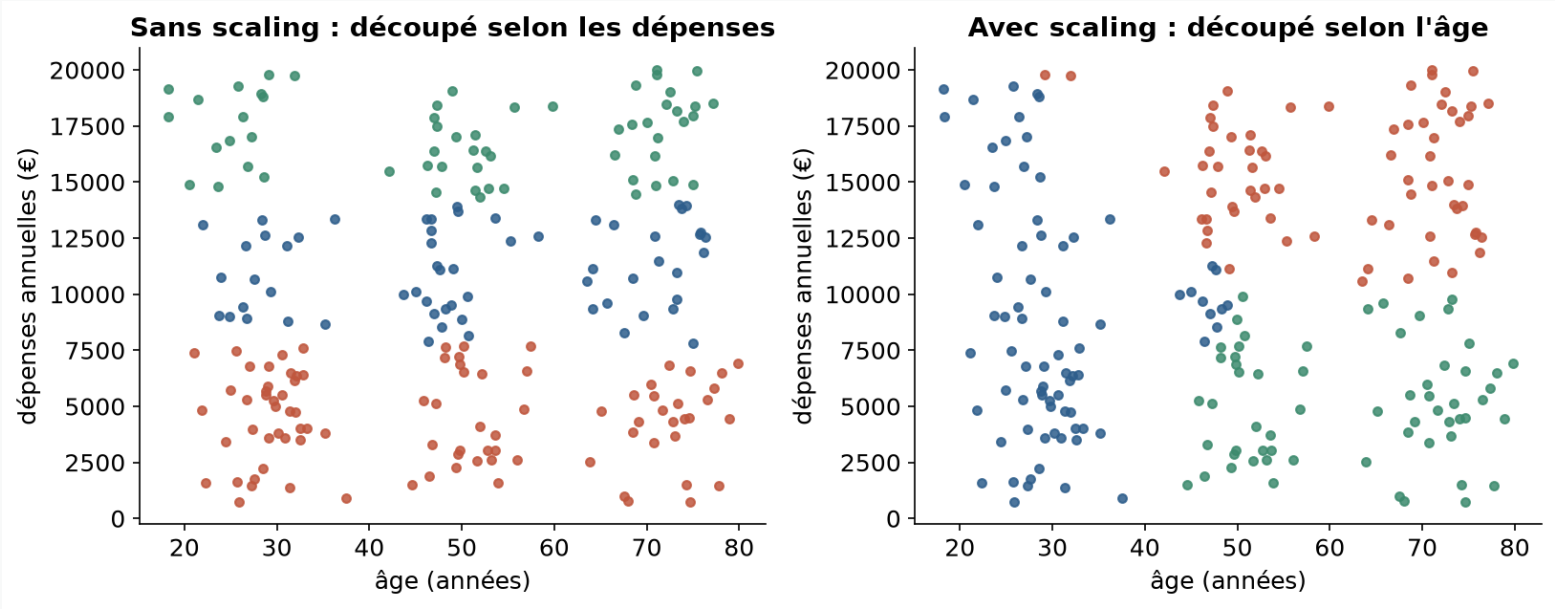
**Rapide** il passe sur de gros volumes : c'est souvent le premier essai.

# Choisir k : deux outils, aucun verdict automatique



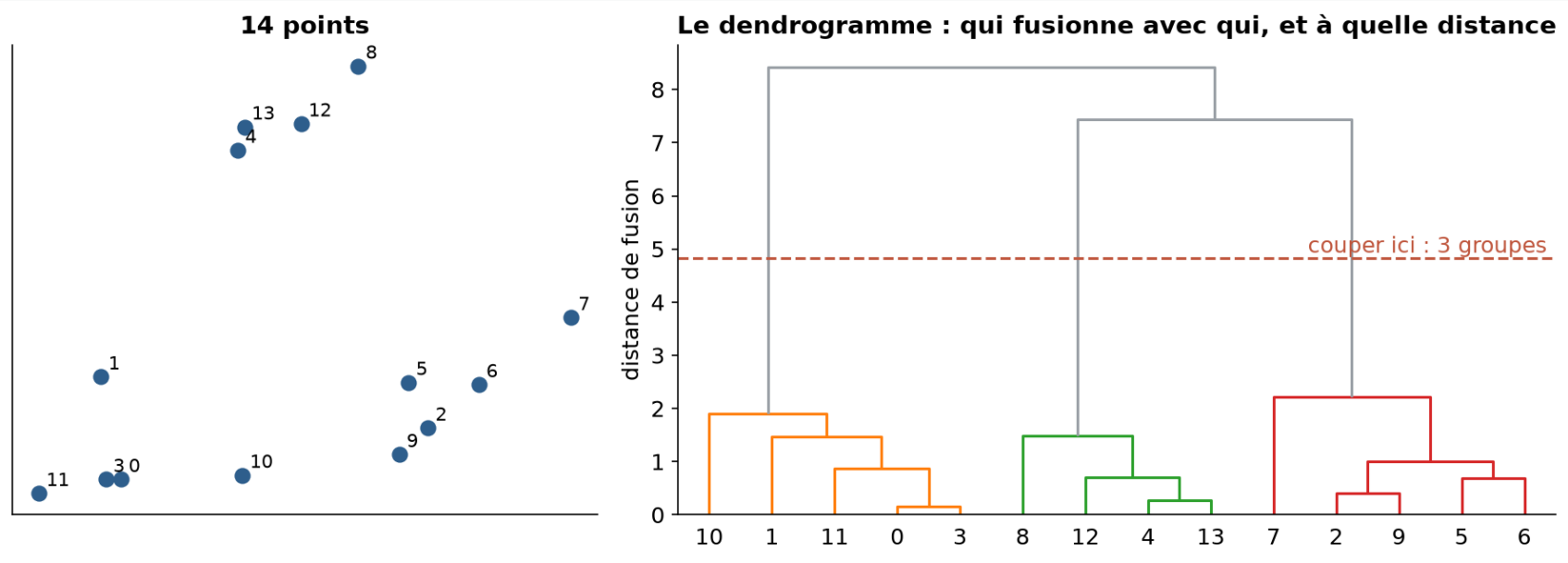

**Elbow method** : l'inertia baisse toujours quand k augmente ; on cherche le « coude » où le gain devient faible. **Silhouette** (de -1 à 1) : chaque point est-il plus proche de son cluster que du cluster voisin ? Ici, les deux indiquent k = 4, parce que les données ont été fabriquées ainsi. Sur des données réelles, ils se contredisent souvent.

# Le scaling décide de ce que « proche » veut dire



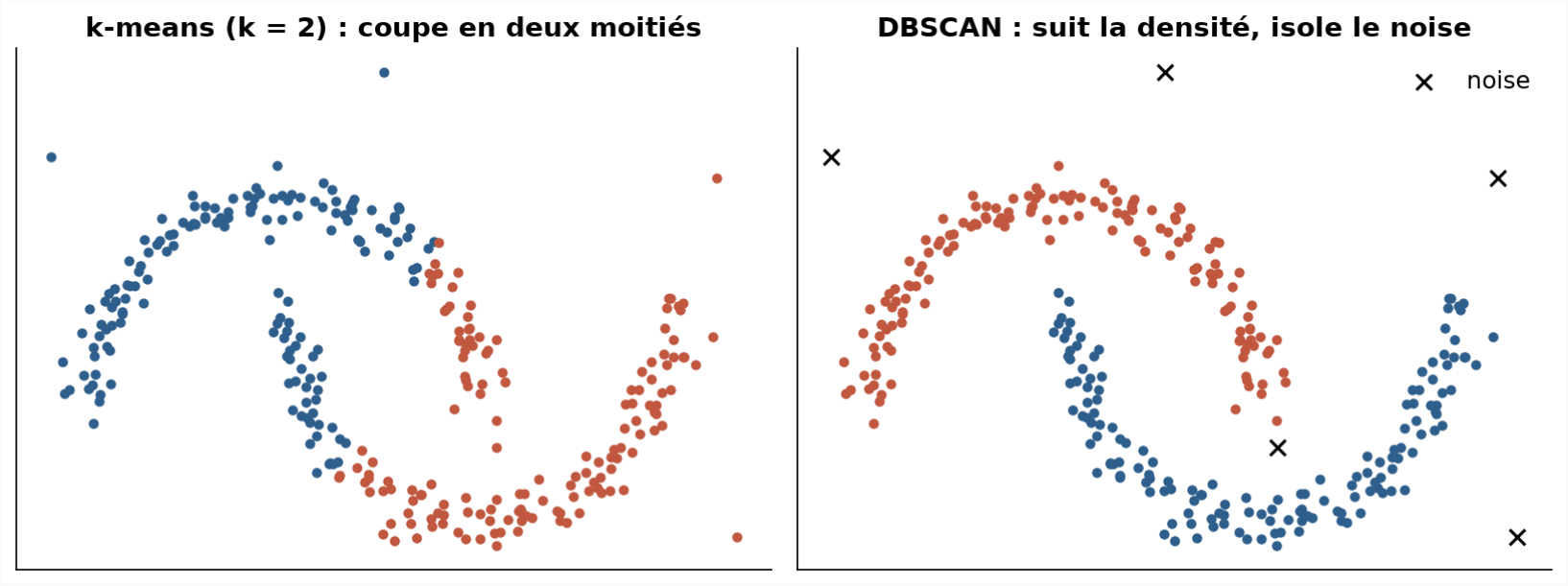

Les mêmes 210 clients. **Sans scaling**, les dépenses (des milliers d'euros) écrasent l'âge (des dizaines d'années) : k-means découpe selon les dépenses. **Avec scaling** (StandardScaler), les deux features pèsent autant et la vraie structure, trois tranches d'âge, apparaît. Changer le scaling, c'est changer la question posée.

# Clustering hiérarchique : le dendrogramme




On part de chaque point seul et on fusionne pas à pas les deux groupes les plus proches. Le **dendrogramme** garde toute l'histoire : la hauteur d'une fusion est la distance entre les groupes fusionnés. On choisit le nombre de clusters **après coup**, en coupant l'arbre ; une longue branche verticale signale des groupes bien séparés.

# DBSCAN : des clusters définis par la densité




Un point qui a au moins `min_samples` voisins dans un rayon `eps` est un **core point** ; les zones denses connectées forment un cluster, les points isolés deviennent du **noise**. Pas de k à choisir, et des formes quelconques. En contrepartie, `eps` est délicat à régler et des densités très différentes posent problème.

# Quel algorithme pour quelle situation ?

| | k-means | Hiérarchique | DBSCAN |
|---|---|---|---|
| Ce qu'on choisit | k | linkage, puis où couper | eps, min_samples |
| Forme des clusters | ronds, tailles proches | selon le linkage | quelconque |
| Points aberrants | affectés quand même | affectés quand même | isolés comme noise |
| Volume de données | très grand | petit à moyen | moyen à grand |
| Atout | simple et rapide | toutes les échelles d'un coup | formes libres, noise |

*Bon réflexe : commencer par k-means, regarder le résultat, puis changer de définition du cluster si la forme des groupes l'exige.*

# Évaluer un clustering sans target

**Métriques internes** **silhouette**, Davies-Bouldin : séparation et compacité des groupes, sans vérité terrain.

**Stabilité** autre graine, autre échantillon, k voisin : les groupes restent-ils les mêmes ?

**Labels, si on en a** l'**ARI** (adjusted Rand index) compare deux partitions : utile pour vérifier, jamais pour entraîner.

**Interprétation métier** décrire chaque cluster (moyennes des features, exemples typiques) : un groupe qu'on ne sait pas décrire ne sert à rien.

**Le piège** une bonne silhouette dit que les groupes sont séparés, pas qu'ils sont **utiles**.

B

# Dimensionality reduction : la PCA

Résumer beaucoup de features en quelques directions, pour voir, compresser et comprendre.

# Pourquoi réduire la dimension ?

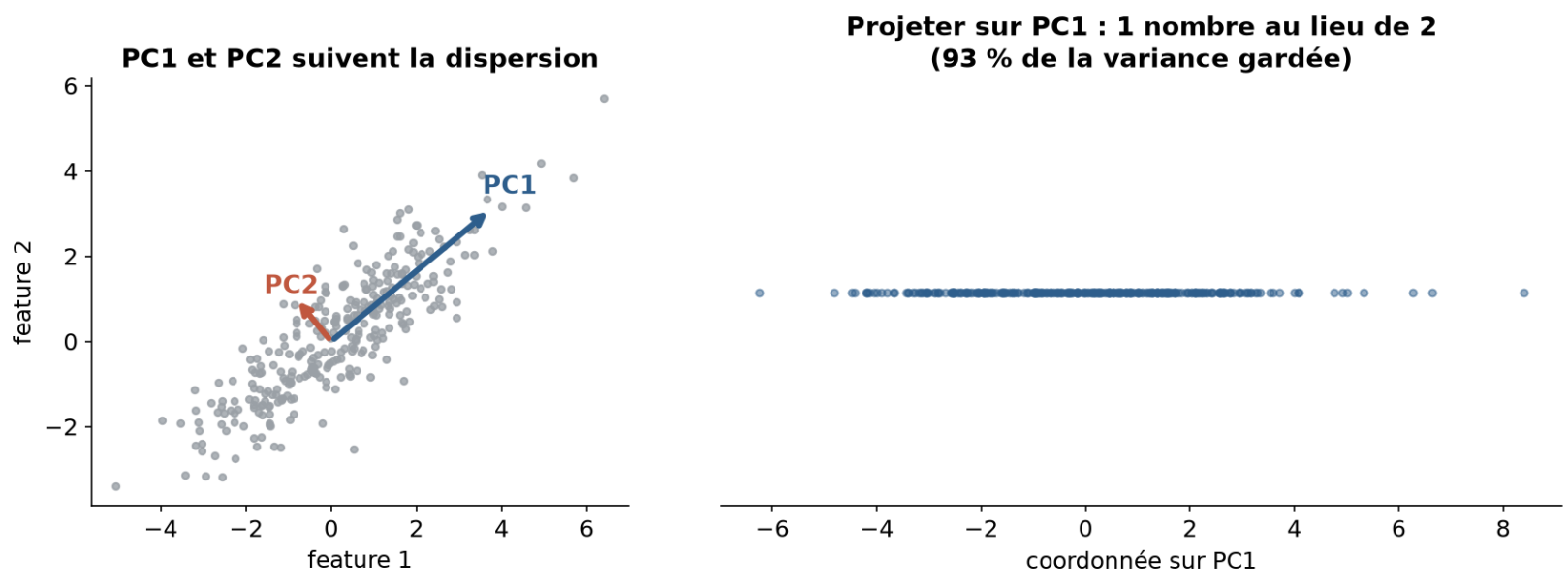
**Voir** on ne peut pas tracer 30 features ; on peut tracer deux composantes.

**Enlever la redondance** beaucoup de features mesurent presque la même chose : rayon, périmètre, aire d'une tumeur.

**Compresser** moins de colonnes, moins de calcul, parfois moins de bruit.

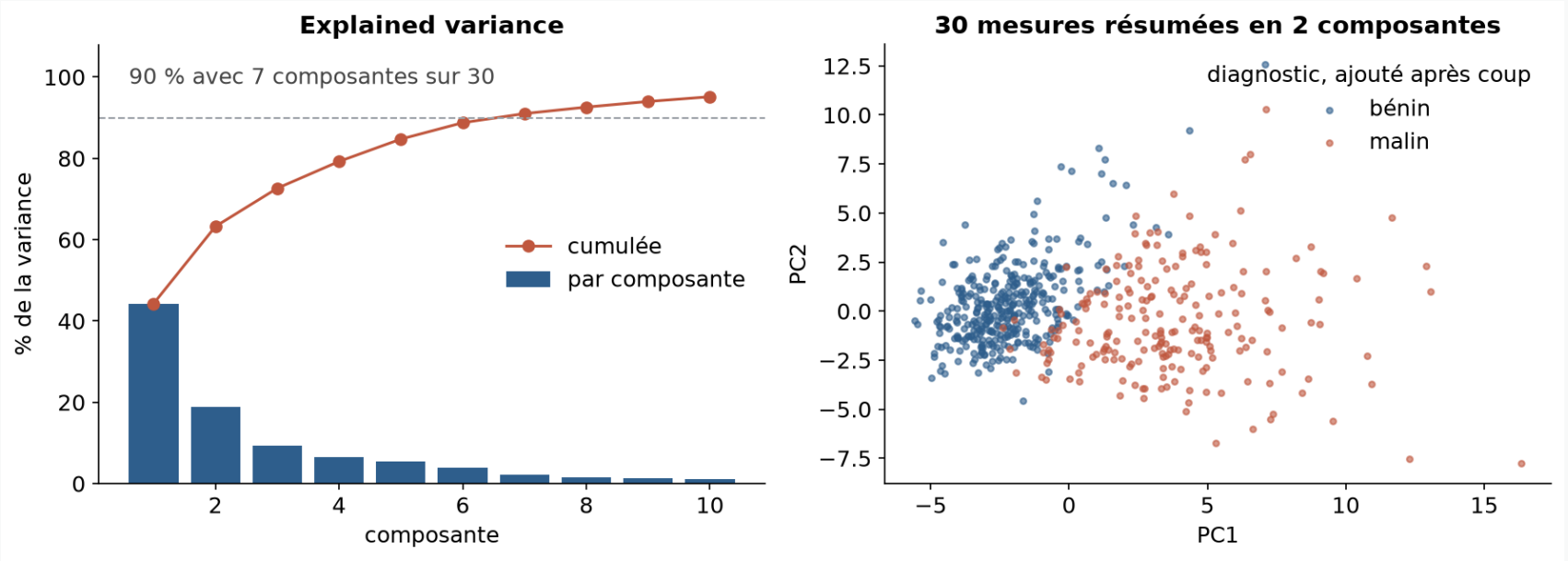
**Curse of dimensionality** en grande dimension, toutes les distances se ressemblent : clustering et plus proches voisins deviennent moins fiables.

# L'idée de la PCA : changer d'axes



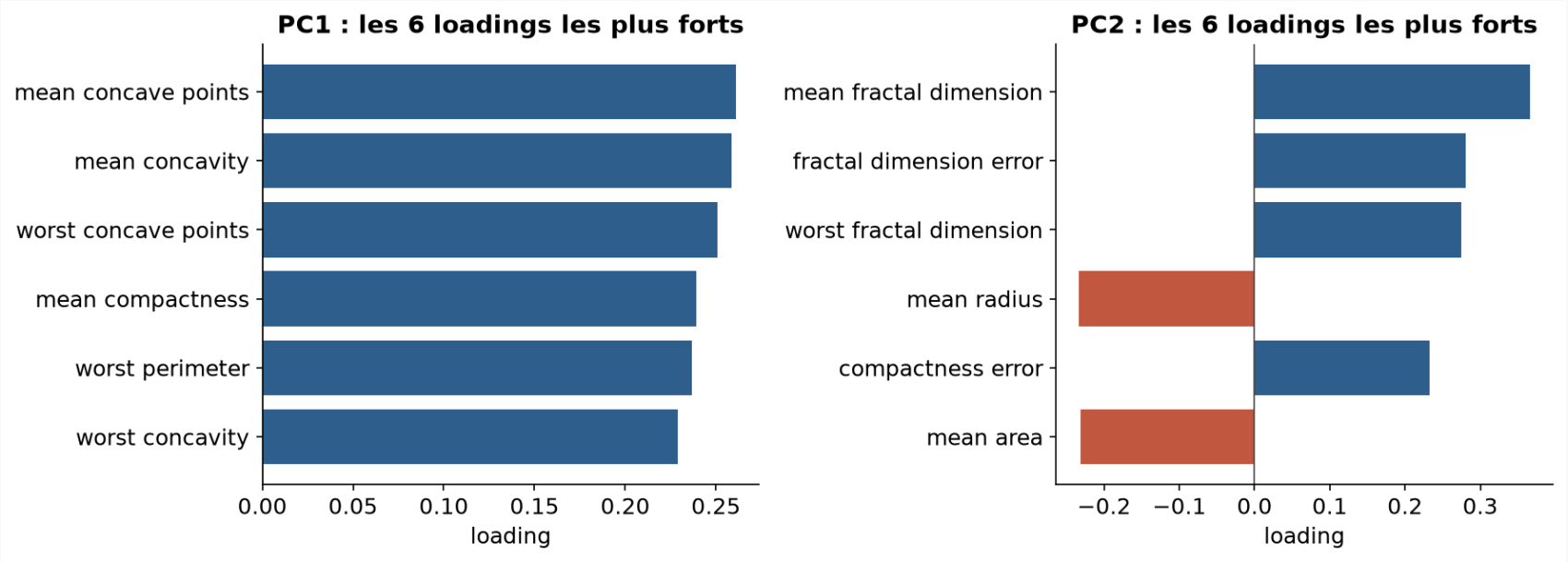

La PCA cherche la direction le long de laquelle les données varient le plus : c'est **PC1**. **PC2** est la suivante, perpendiculaire à la première, et ainsi de suite. Ici les deux features sont très corrélées : projeter sur PC1 garde 93 % de la variance avec un seul nombre au lieu de deux. Une rotation des axes, puis un choix : combien de composantes garder.

# Combien de composantes garder ? L'explained variance




Sur un jeu médical réel (569 tumeurs, 30 mesures), PC1 garde 44 % de la variance et les deux premières 63 % ; il faut 7 composantes sur 30 pour atteindre 90 %. À droite, le diagnostic, **ajouté après coup**, n'a jamais été vu par la PCA : les deux groupes se séparent pourtant bien. Seuils courants : 90 % ou 95 %, ou le coude de la courbe.

# Lire une composante : les loadings




Chaque composante est une **combinaison des features d'origine** ; ses coefficients sont les **loadings**. PC1 pèse sur la concavité, la compacité et le périmètre : un axe « tumeur grande et irrégulière ». PC2 oppose la dimension fractale à la taille. Le signe est arbitraire : seul compte qui va avec qui, et qui s'oppose à qui.

# La PCA : usages et limites

**Standardiser avant** sinon la feature aux plus grands nombres devient PC1 à elle seule.

**Elle ne voit pas la target** la direction qui varie le plus n'est pas forcément celle qui aide à prédire.

**Elle est linéaire** les structures courbes lui échappent ; **t-SNE** et **UMAP** servent à visualiser, pas à modéliser.

**Moins lisible** une composante mélange toutes les features : on perd le sens direct des colonnes.

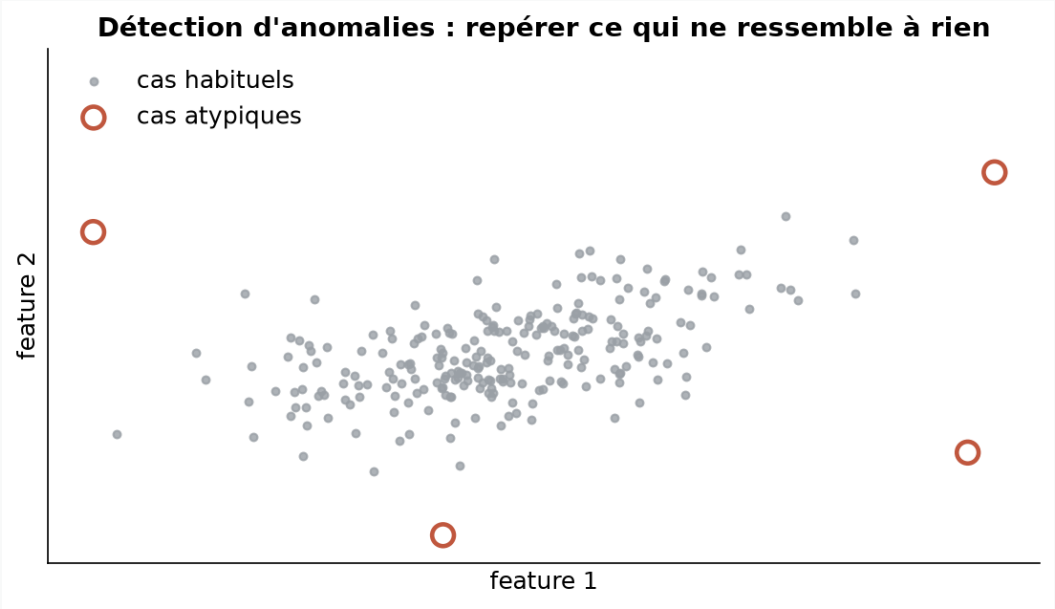
**Dans un pipeline** fit sur le train set seulement, comme tout preprocessing, sinon **data leakage**.

C

# Anomalies et mise en pratique

Repérer l'inhabituel, puis les réflexes d'un projet non supervisé.

# Anomaly detection : repérer l'inhabituel




On apprend à quoi ressemble le **normal**, et on signale ce qui s'en écarte. Les méthodes déjà vues s'y prêtent : un point loin de tout centre k-means, un point classé noise par DBSCAN. Des méthodes dédiées existent, comme **Isolation Forest** ou Local Outlier Factor. Usages : fraude, panne de capteur, erreur de saisie, cas clinique atypique.

# Les pièges classiques

**Il y a toujours une réponse** demandez 4 clusters à du bruit pur : vous aurez 4 clusters. L'algorithme ne dit jamais « il n'y a rien ».

**Le scaling choisit pour vous** sans standardisation, la colonne aux plus grands nombres décide.

**k est une décision** le bon nombre de groupes dépend de l'usage, pas seulement d'une métrique.

**Un cluster est une hypothèse** pas une vérité : il faut le décrire, le tester sur d'autres données, le faire valider par le métier.

**Les features décident** changer de features, c'est changer de question, donc de groupes.

# Un projet non supervisé en cinq étapes

*Le cycle d'un projet de data science, avec une étape d'interprétation plus lourde, faute de target pour trancher.*

1

## Question

à quoi serviront les groupes ou la projection ?

›

2

## Features

choisir, nettoyer, encoder, **scaler**.

›

3

## Méthode

k-means d'abord, une autre définition si besoin.

›

4

## Évaluer

silhouette, stabilité, autre k, autre graine.

›

5

## Interpréter

décrire chaque groupe, le faire valider par le métier.

# Où le rencontre-t-on ?

**Marketing** segmenter des clients selon leurs achats pour adapter les offres.

**Santé** trouver des sous-groupes de patients aux profils proches, repérer des parcours de soins atypiques.

**Industrie** surveiller des capteurs et détecter une panne avant qu'elle n'arrive.

**Texte et images** regrouper des documents par thème, visualiser des embeddings en 2D.

**Avant un modèle supervisé** explorer les données, créer une feature « cluster », réduire la dimension.

# Ce qu'il faut retenir

| Méthode | Question posée | À choisir | Vigilance |
|---|---|---|---|
| k-means | quels groupes autour de centres ? | k | scaling, formes rondes |
| Hiérarchique | quels groupes, à quelle échelle ? | linkage, hauteur de coupe | lent sur gros volumes |
| DBSCAN | quelles zones denses, quel noise ? | eps, min_samples | densités inégales |
| PCA | quelles directions résument les données ? | nombre de composantes | standardiser, ne voit pas la target |
| Anomaly detection | qu'est-ce qui est inhabituel ? | seuil d'alerte | le « normal » doit être représentatif |

# Trois questions à poser devant une segmentation

*On vous en montrera bien plus souvent que vous n'en construirez.*

**1 Quelles features, et avec quel scaling ?** Sans réponse, les segments peuvent n'être qu'un artefact de la colonne aux plus grands nombres.

**2 Quelle est la force de la structure ?** « Nous avons trouvé quatre segments » ne dit pas s'ils sont nets ou arbitraires.

**3 Que se passe-t-il avec trois segments, ou cinq ?** Si la conclusion change à chaque k, elle repose sur un choix que personne n'a défendu.

# Contrôle des connaissances

1. Quelle est la différence fondamentale entre un problème supervisé et un problème non supervisé ?

2. Pourquoi faut-il standardiser les features avant k-means ou une PCA ?

3. L'inertia baisse quand k augmente. Pourquoi ne pas prendre le k qui la minimise ?

4. Sur deux croissants imbriqués, quel algorithme choisir, et pourquoi ?

5. Une PCA garde 90 % de la variance. Le modèle supervisé sera-t-il forcément aussi bon ?

# Si une seule chose doit rester :

Une méthode non supervisée renvoie toujours une réponse. Votre travail est de vérifier si cette réponse est **solide**, et si elle est **utile**.

**Pour aller plus loin : notebooks 90 (clustering) et 91 (PCA), en auto-formation**